Cross-Domain Evaluation of Diamond-DETR for Surface-Defect Detection

Transformers versus CNNs on the Magnetic Tile Defect Dataset

Deep Vision, Topic 25: Comparison of Transformers and CNNs

Adarsh Koshiya

Ostbayerische Technische Hochschule (OTH) Amberg-Weiden

Deep Vision, Computer Vision and AI · Sommersemester 2026

Abstract. We ask whether the architectural choices behind *Diamond-DETR*, a transformer detector designed for synthetic-diamond surface inspection, transfer to a different industrial domain: magnetic tile surface defects. Four detectors are trained and evaluated under matched conditions on a five-class defect task (blowhole, break, crack, fray, uneven) with severe class imbalance. Under transfer learning the transformer RT-DETR-r18 reaches a test mAP@0.5 of 0.763, nearly double the best CNN (YOLOv8n, 0.403), and, more importantly, spreads accuracy evenly across all five classes (per-class mAP@[0.5:0.95] from 0.38 to 0.58) where both CNNs collapse on the rare *break* and *fray* defects (AP $\approx$ 0.02 to 0.08). The transformer recall of 0.82 against the CNNs' 0.23 to 0.35 is the decisive property for inspection, where a missed defect ships a faulty part. The cost is speed (40 against 150 to 205 FPS) and size (19.9 M against 2.6 to 3.0 M parameters). On the central architectural question, Diamond-DETR matches RT-DETR on mAP@0.5 (0.762 against 0.763) and exceeds it on the stricter mAP@[0.5:0.95] (0.489 against 0.446) at 28% fewer parameters and 40% higher speed, and it does so from *random initialisation* while RT-DETR had a diamond-domain pretraining head start. The paper's "lightweight yet stronger" claim therefore holds under cross-domain transfer, with the honest caveat that the comparison is not initialisation-controlled.

Contents

1 Introduction and research question **2**

2 Background **2**

3 Dataset and preparation **2**

4 Models compared and provenance **3**

4.1 How each model works, what trains it, and where it comes from 4

5 Experimental setup **4**

6 Results **5**

7 Discussion **7**

7.1 Transformers against CNNs 7

7.2 Diamond-DETR against RT-DETR 8

8 Grad-CAM attention analysis **8**

9 Cross-domain comparison **9**

10 Limitations **10**

11 Future work **10**

12 Conclusion **10**

A Data integrity and engineering notes **11**

B Fork compatibility patches **11**

References **12**

1 Introduction and research question

Surface-defect detection is a quality-critical task in industrial inspection: the dominant cost is a *missed* defect, since a false alarm only triggers a re-check while an undetected flaw ships a faulty part. This asymmetry makes recall, and balanced performance across rare defect types, matter more than a single aggregate accuracy number.

Diamond-DETR [1] is a transformer detector published for synthetic-diamond surface inspection. It modifies the real-time detection transformer RT-DETR [2] with three lightweight components and claims to be "lightweight yet at least as strong" as its baseline. This study tests whether that claim, and more generally the transformer advantage, transfers to a different industrial domain: the Magnetic Tile Surface Defect dataset [4].

We compare two transformer detectors (Diamond-DETR and RT-DETR-r18) against two compact CNN detectors (YOLOv8n and YOLO11n) under matched training conditions, and ask:

1. Do transformer detectors outperform compact CNN detectors on small, imbalanced industrial defect data, and where does any advantage come from?
2. Does Diamond-DETR's advantage over its own RT-DETR baseline survive the domain change?

The narrative and analysis structure of this report follow the project notebook (`MagTile_DiamondDETR_Notebook`) we frame the problem as five-class detection, convert segmentation masks to boxes, train under one harness, and read all final metrics from machine-written JSON artifacts so that no number in this document is entered by hand.

2 Background

RT-DETR [2] is a real-time detection transformer: a ResNet backbone feeds an efficient hybrid encoder (the *AIFI* attention module on the deepest feature map plus a CNN-based cross-scale fusion), and a transformer decoder performs *set prediction*. Following DETR [3], the decoder emits one query per object, is matched to ground truth by a bipartite (Hungarian) assignment, and requires *no* non-maximum suppression. This global, NMS-free formulation is the structural difference from anchor-based CNN heads.

Diamond-DETR [1] keeps RT-DETR's AIFI encoder and decoder head and swaps three components, summarised in Table 2: a re-parameterisable FasterNet backbone block, a high-level screening feature-pyramid neck with channel attention, and a re-parameterisable fusion block. The intent is higher accuracy at fewer parameters.

YOLOv8n and YOLO11n [5] are compact anchor-free single-stage CNN detectors (C2f and C3k2 backbones respectively, PAN-style necks, decoupled detection heads). They are the established and current real-time baselines.

3 Dataset and preparation

The Magnetic Tile Surface Defect dataset [4] contains 1,344 grayscale tile images with five defect types plus a large defect-free class. We frame the task as five-class detection (literature precedent exists), with the 952 defect-free tiles acting as negatives with empty label files. Table 1 shows the class distribution; the imbalance is severe, with *fray* represented by only 32 images.

Table 1. Class distribution of the Magnetic Tile dataset. The imbalance, especially the 32 fray images, motivates a per-class analysis rather than a single aggregate metric.

	blowhole	**break**	**crack**	**fray**	**uneven**	**defect-free**
Images	115	85	57	32	103	952
Box instances	118	117	70	38	105	0 (negatives)

Masks to boxes (the data philosophy). The dataset ships pixel segmentation masks, not boxes. We convert each mask to YOLO-format boxes by connected-component analysis: one box per connected defect region with area at least 20 px, the box taken as the component's bounding rectangle. A whole-image fallback box is emitted only when a mask yields no region, which fired on just 6 of 392 defect images. This step is deliberately lossy in one way: two touching defect regions can merge into a single box, which is recorded as a limitation (Section 10).

Split. A stratified 70/20/10 split with seed 42 produces 938 training, 268 validation, and 138 test images, with zero image overlap between splits (verified). All accuracy numbers in this report are measured on the held-out test split.

4 Models compared and provenance

Table 2. The four detectors. Diamond-DETR is RT-DETR-r18 with three components swapped (B, C, D); it shares the AIFI encoder and the set-prediction decoder head with RT-DETR.

Model	**Family**	**Params**	**Role in the study**
Diamond-DETR	Transformer	14.3 M	The paper model under test
RT-DETR-r18	Transformer	19.9 M	The baseline Diamond-DETR improves upon (isolates its contribution)
YOLOv8n	CNN	3.0 M	Established real-time CNN detector
YOLO11n	CNN	2.6 M	Current-generation CNN detector

Diamond-DETR's three swaps, relative to RT-DETR-r18, are:

- **(B)** `BasicBlock_Faster_Block_Rep` backbone blocks (re-parameterisable FasterNet convolutions) replacing the standard ResNet-18 `BasicBlock`, at the same channel dimensions (64/128/256/512);
- **(C)** `ChannelAttention_HSFPN`, a channel-attention high-level screening feature pyramid, replacing the standard feature-pyramid neck;
- **(D)** `DRBC3`, a re-parameterisable fusion block, replacing the `RepC3` fusion blocks.

What is reused versus built here. The four model architectures, the forked Ultralytics 8.0.201 that defines the DETR custom modules, RT-DETR's diamond-pretrained checkpoint, and the COCO-pretrained YOLO weights are reused from the upstream repositories [1, 5]. Built for this study are the data-preparation script (mask to box plus stratified split), the training harness, the metrics writer, six compatibility patches that let the unmaintained fork run on modern PyTorch and NumPy, and the Grad-CAM analysis. The trained checkpoints and every test-split number in Section 6 were produced and measured here.

4.1 How each model works, what trains it, and where it comes from

Table 3 is the provenance map: for each detector it states the mechanism, the script and recipe that trained it, and the weights it started from. The two transformers share the AIFI attention encoder and the `RTDETRDecoder` set-prediction head (one query per object, Hungarian matching, no NMS); they differ only in backbone and neck. All four are evaluated on the test split by `evaluate.py` (`--type rtdetr` for the DETR models, which require the fork on `PYTHONPATH`; `--type yolo` for the CNNs).

Table 3. Per-model mechanism, training recipe, and initialisation. "train.sh, fork" is the forked Ultralytics 8.0.201 run from `code/diamond_detr/RTDETR-main`; "stock" is the pip-installed Ultralytics 8.4.x.

Model	How it works (mechanism)	Trained by	Initialised from
Diamond-DETR	RT-DETR-r18 with three swapped components: RepFasterNet backbone (B), HS-FPN channel-attention neck (C), `DRBC3` fusion (D); shared AIFI encoder and decoder.	train.sh, fork; AdamW, 200 ep, batch 4	Random init (authors published no weights)
RT-DETR-r18	ResNet-18 backbone (`ConvNormLayer` + `BasicBlock`), standard FPN + `RepC3` neck, AIFI encoder, `RTDETRDecoder` head.	train.sh, fork; AdamW, 200 ep, batch 4	Paper's `best.pt` (diamond-pretrained, mAP@0.5 0.89); partial load via `intersect_dicts`, head re-init for nc=5
YOLOv8n	Anchor-free single-stage CNN: C2f backbone, PAN-FPN neck, decoupled head with DFL.	train.sh, stock; SGD, 150 ep, batch 4	`yolov8n.pt` (COCO-pretrained)
YOLO11n	Newer Ultralytics CNN: C3k2 backbone, C2PSA attention block, decoupled head.	train.sh, stock; SGD, 150 ep, batch 4	`yolo11n.pt` (COCO-pretrained)

5 Experimental setup

Both families share the image size (640×640), batch size (4), seed (42), and a learning rate of 10^{-4}. They differ in the two variables that model-family practice dictates, summarised in Table 4. Transformer detectors require AdamW: an initial SGD run left both transformers at approximately zero mAP, since SGD at this learning rate does not move attention weights. The pretrained CNNs converge by 150 epochs; the transformers were given 200.

Table 4. Training configuration. The optimiser and epoch-count differences are deliberate and follow model-family practice; they are discussed as a confound in Section 10.

Setting	CNNs (YOLOv8n, YOLO11n)	Transformers (RT-DETR, Diamond-DETR)
Optimiser	SGD ($\mu = 0.8$)	AdamW ($\mu = 0.9$)
Epochs	150	200
Batch / image	4 / 640	4 / 640
Learning rate	10^{-4}	10^{-4}
Initialisation	COCO-pretrained	RT-DETR: diamond-pretrained; Diamond-DETR: random

Two-environment architecture. The DETR family runs on a forked Ultralytics 8.0.201 (which defines the custom modules), and the YOLO family runs on stock Ultralytics 8.4.x. The two versions cannot share one Python process, so the training harness runs each model in a separate subprocess and writes test metrics to JSON; the notebook only reads those JSON files. Six compatibility patches were applied to the fork (for example `np.trapz` to `np.trapezoid`, and a `torch.load` default change), all of which are compatibility fixes, not model changes.

Initialisation note. RT-DETR-r18 was initialised from the paper's own checkpoint, trained on a two-class diamond task; its backbone and neck transfer through a partial load while the five-class head is re-initialised. Diamond-DETR has no public pretrained weights anywhere in the repository, so it trains from *random initialisation*. It converged regardless (validation mAP@0.5 climbed from ≈ 0.00001 to 0.80 over 200 epochs).

6 Results

Table 5 reports the headline test-split metrics and Figure 1 plots the two aggregate accuracies. Table 6 and Figure 2 give the per-class breakdown, which is the core finding of the study. Figure 3 places the four models on the accuracy versus speed plane.

Table 5. Test-split results (138 images, seed 42). Best value per column in bold. Transformers use AdamW for 200 epochs; CNNs use SGD for 150 epochs.

Model	mAP@0.5	mAP@[.5:.95]	Precision	Recall	FPS	Params
Diamond-DETR (random init)	0.762	0.489	0.745	0.746	56.5	**14.3 M**
RT-DETR-r18 (pretrained)	0.763	0.446	0.733	0.819	40.2	19.9 M
YOLOv8n (COCO)	0.403	0.275	0.613	0.349	204.7	3.0 M
YOLO11n (COCO)	0.335	0.204	0.579	0.227	153.2	2.6 M

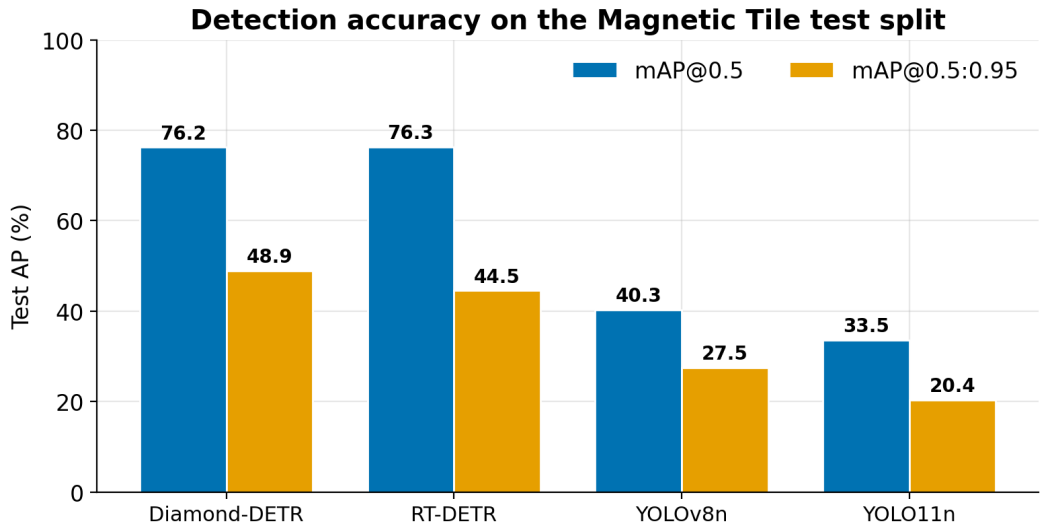


Figure 1. Aggregate detection accuracy. Both transformers reach roughly 0.76 mAP@0.5; the CNNs sit near 0.34 to 0.40. Diamond-DETR leads on the stricter mAP@[0.5:0.95].

Table 6. Per-class AP@[0.5:0.95] on the test split. The CNNs are competitive only on the large, high-contrast *blowhole* class and nearly collapse on the rare, elongated classes; both transformers keep every class at or above 0.36.

Model	blowhole	break	crack	fray	uneven
Diamond-DETR	0.670	0.502	0.536	0.383	0.356
RT-DETR-r18	0.578	0.415	0.388	0.463	0.383
YOLOv8n	0.707	0.074	0.280	0.069	0.244
YOLO11n	0.563	0.078	0.175	0.023	0.181

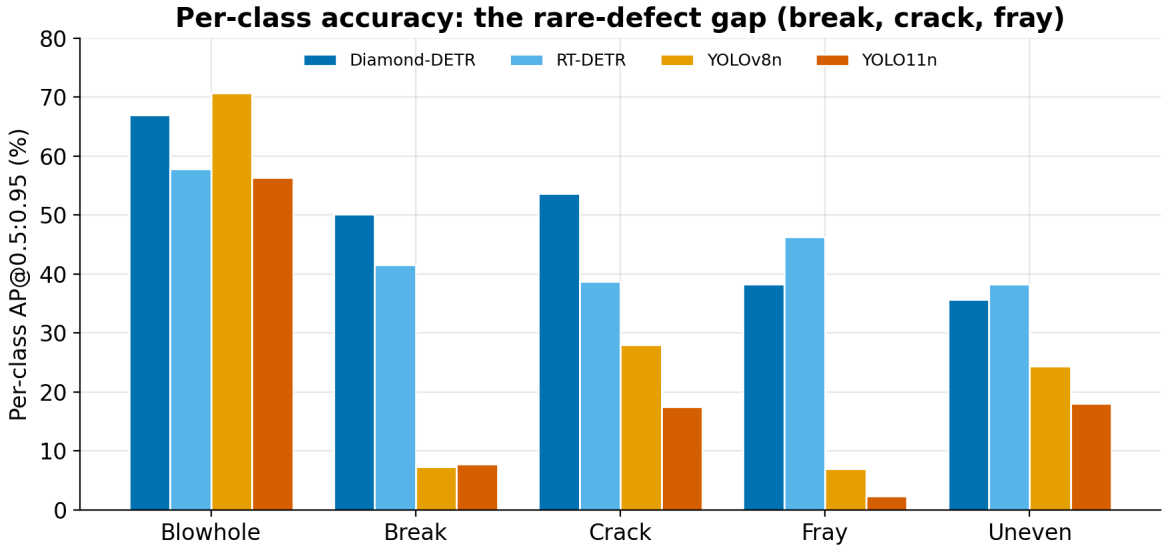


Figure 2. Per-class accuracy. On the rarest class *fray* (32 training images) RT-DETR scores 0.463 against YOLOv8n's 0.069, a factor of about 6.7, and against YOLO11n's 0.023, a factor of about 20. This is the headline of the study: the transformers convert class imbalance from a near-total failure into a manageable spread.

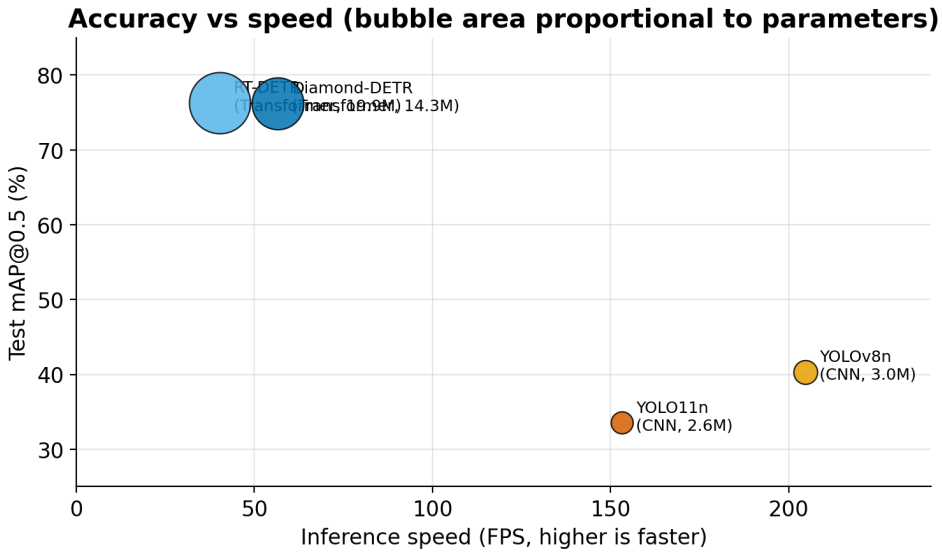


Figure 3. Accuracy against speed; bubble area is proportional to parameter count. The transformers buy accuracy and balanced recall at roughly five times the latency and seven times the parameters of the CNNs, yet remain real-time at 40 to 56 FPS.

Validation against test is consistent: RT-DETR reached validation mAP@0.5 0.731 at epoch 200 and test 0.763; Diamond-DETR reached validation 0.797 and test 0.762. The two transformers land at the same mAP@0.5, but Diamond-DETR wins the stricter mAP@[0.5:0.95] (0.489 against 0.446), that is, tighter boxes, at 28% fewer parameters and 40% higher FPS. In wall-clock terms the per-image latencies are roughly 18 ms (Diamond-DETR), 25 ms (RT-DETR), 5 ms (YOLOv8n), and 7 ms (YOLO11n), so every model clears real-time for line-scan inspection.

7 Discussion

7.1 Transformers against CNNs

RT-DETR reaches a test mAP@0.5 of 0.763 against YOLOv8n's 0.403, a factor of 1.9. Three observations matter more than that headline.

Recall, not precision, is where the transformer wins. Precision is broadly comparable (RT-DETR 0.73 against YOLOv8n 0.61), but recall jumps from 0.35 (YOLOv8n) and 0.23 (YOLO11n) to 0.82. The CNNs are conservative: when they fire they are often right, but they miss roughly two thirds of defects. For inspection this is the costly failure mode. The set-prediction head (one query per object, Hungarian matching, no NMS) recovers defects the anchor-based heads drop.

Only the transformers learn the rare classes. Both CNNs essentially solve *blowhole* (large, round, high-contrast) and then fall off a cliff on the elongated, scarce classes. Both transformers keep every class at or above 0.36 (Table 6). Two causes plausibly compound: the AIFI attention encoder supplies global context that helps localise thin, low-contrast cracks and frays that a limited CNN receptive field struggles with, and pretraining supplies a feature prior that 32 fray images cannot teach from scratch.

Cost. The accuracy comes at roughly five times the latency (40 against 205 FPS) and seven times the parameters (19.9 against 3.0 M). At 40 FPS the transformer still clears real-time for line-scan inspection, so for a quality-critical task the trade is worth it here.

Confound, read the win carefully. RT-DETR was initialised from a diamond-domain checkpoint while the CNNs were initialised from COCO. Part of RT-DETR's advantage is therefore domain-relevant pretraining, not architecture alone. The balanced per-class behaviour is partly

architectural (attention plus set matching) and partly this transfer; the two cannot be fully separated with the current runs.

7.2 Diamond-DETR against RT-DETR

On the central architectural question, Diamond-DETR matches RT-DETR on mAP@0.5 (0.762 against 0.763) and beats it on mAP@[0.5:0.95] (0.489 against 0.446) while being lighter (14.3 against 19.9 M) and faster (56 against 40 FPS). On the three hardest classes it is the single best model (*break* 0.502, *crack* 0.536, *blowhole* 0.670). This reproduces the paper's "lightweight yet stronger" claim under cross-domain transfer. Three points sharpen it.

First, the win is achieved from a worse starting point: Diamond-DETR trained from random init while RT-DETR was pretrained, so the parity is conservative rather than inflated. Second, the equal mAP@0.5 with higher mAP@[0.5:0.95] means Diamond-DETR finds the same defects as RT-DETR but localises them more tightly, consistent with its sharper multi-scale fusion; RT-DETR keeps the higher recall (0.819 against 0.746), so the two are complementary. Third, the single most important methodological lesson of the project is that transformer detectors require AdamW: the initial SGD run would have wrongly concluded that "CNNs beat transformers".

8 Grad-CAM attention analysis

To corroborate the quantitative gap, Grad-CAM [6] attention maps were computed for each trained detector on one tile per defect class, using the official `pytorch-grad-cam` library on the deepest backbone convolution. Figure 4 shows the comparison grid.

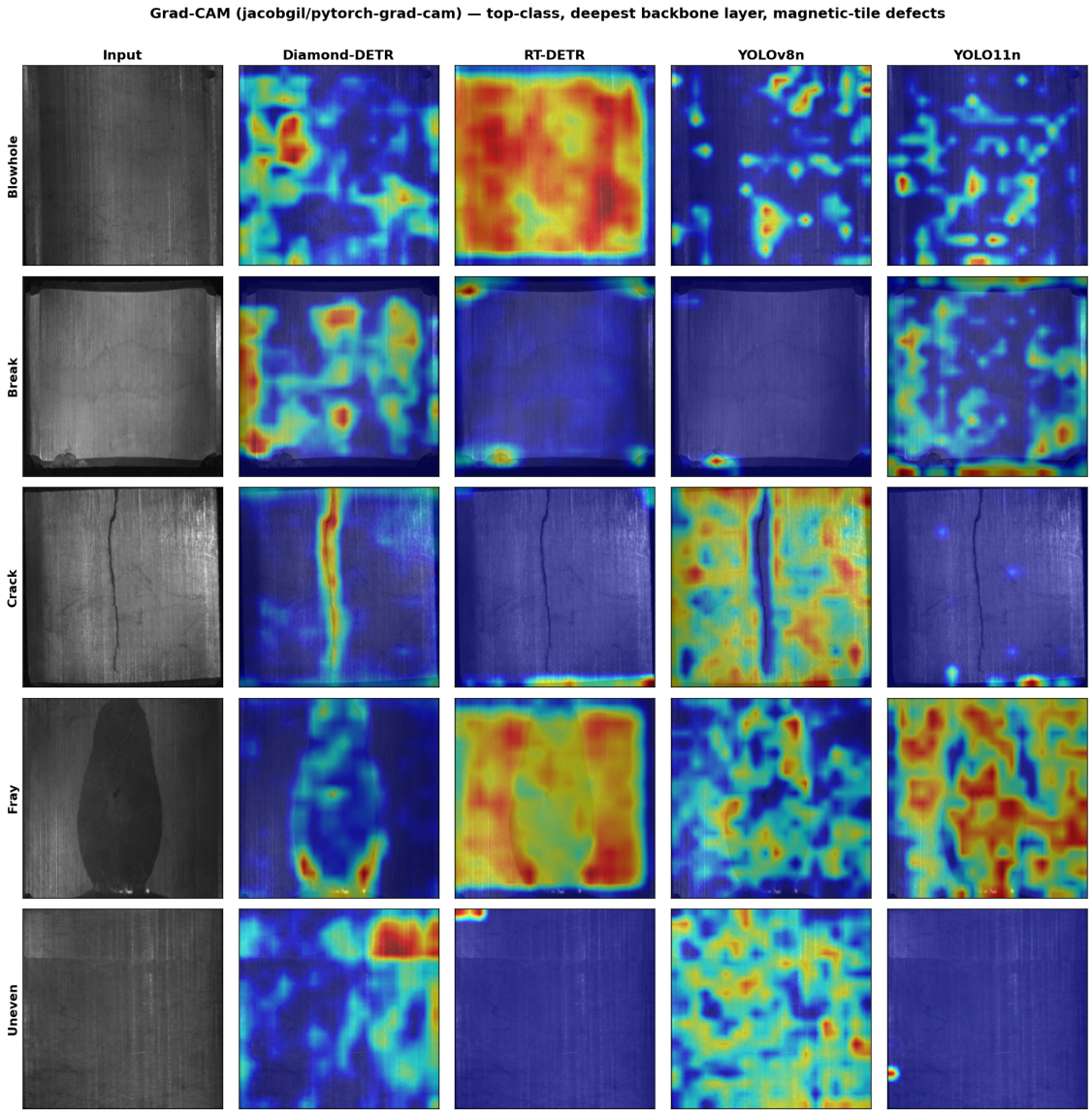


Figure 4. Grad-CAM attention overlays. Rows are the five defect classes; columns are the input and the four models. The transformers, especially Diamond-DETR, produce focused, defect-aligned maps (a sharp line along the *crack*), while the CNN maps are diffuse. The top predicted class also separates them: both transformers classify all five defects correctly, whereas YOLO11n collapses to class 0 on almost everything, mirroring its near-zero rare-class AP.

9 Cross-domain comparison

Table 7 places our magnetic-tile results beside the values reported for the original diamond domain. The drop from the diamond figures to the tile figures is expected (a different, smaller, more imbalanced domain), but the *relative* story is preserved: the two transformers remain close to one another and far ahead of the CNN, and Diamond-DETR remains competitive with RT-DETR while being lighter. The diamond-domain figures are quoted for reference and should be verified against the source paper before citation.

Table 7. Cross-domain comparison. "Diamond (paper)" figures are reference values from the source paper [1] and are to be verified; "Magnetic Tile (ours)" are this study's test-split metrics from `results/metrics/*.json`. Values are percentages.

Model	Domain	P	R	mAP@0.5	mAP@[.5:.95]
Diamond-DETR	Diamond (paper, ref.)	92.8	88.3	90.6	n/a
	Magnetic Tile (ours)	74.5	74.6	76.2	48.9
RT-DETR	Diamond (paper, ref.)	89.7	87.9	88.0	n/a
	Magnetic Tile (ours)	73.3	81.9	76.3	44.6
YOLOv8n	Diamond (paper, ref.)	88.7	87.2	83.2	n/a
	Magnetic Tile (ours)	61.3	34.9	40.3	27.5

10 Limitations

1. **Not initialisation-controlled.** Diamond-DETR trains from random init while RT-DETR is pretrained on diamonds. The asymmetry favours RT-DETR, so Diamond-DETR's parity is a conservative result, but a fully clean test would pretrain both identically.
2. **Different optimisers and epoch counts per family** (AdamW/200 against SGD/150). These are justified by model-family practice but are not controlled variables.
3. **Framework confound.** Transformers run on a forked Ultralytics 8.0.201, CNNs on stock 8.4.x; hyper-parameters were matched manually.
4. **Mask to box is lossy.** Touching defect regions can merge into a single box.
5. **Class imbalance.** Fray has 32 training images; per-class AP is therefore essential and aggregate mAP alone would be misleading.
6. **Single seed** (42). Mean and standard deviation over several seeds would be needed to claim that the 0.762 against 0.763 difference is significant; on a 138-image test split it is almost certainly within noise.

11 Future work

The highest-value follow-up is a from-scratch RT-DETR-r18 baseline under the identical recipe, which removes the only confound in the central architectural comparison and makes Diamond-DETR against RT-DETR fully initialisation-controlled. Next are a component ablation that isolates the three swaps (B, C, D), and multi-seed runs with bootstrap confidence intervals so that small differences can be called significant or not. Beyond that: a true multi-domain evaluation on NEU-DET [10] and similar datasets; class-imbalance methods targeted at the rare defects, such as focal loss [7], class-balanced re-weighting [8], and copy-paste augmentation [9]; an instance-segmentation framing that uses the original masks and reports mask IoU; and deployment work (quantisation, distillation) to map a fuller accuracy versus latency frontier. The full, prioritised list is in `docs/FUTURE_WORK.md`.

12 Conclusion

On the Magnetic Tile defect dataset the transformer detector RT-DETR-r18 clearly outperforms both compact CNNs (test mAP@0.5 0.763 against 0.403 and 0.335) and, decisively for inspection, raises recall to 0.82 against 0.23 to 0.35 with balanced per-class accuracy where the CNNs learn only the easy blowhole class. On the study's central question, Diamond-DETR's three component swaps improve on its RT-DETR baseline in efficiency and localisation: equal mAP@0.5, higher

mAP@[0.5:0.95], at 28% fewer parameters and 40% higher speed, and from a random-initialisation disadvantage. Two honest caveats remain: the comparison is not initialisation-controlled, and the key practical lesson is that transformer detectors require AdamW, since an initial SGD run left both at approximately zero mAP and would have produced the opposite, wrong conclusion.

A Data integrity and engineering notes

The study was rebuilt several times as problems were found and fixed. The episodes below are recorded both for honesty and because they explain non-obvious choices in the main text.

- **Data leakage caught and removed.** An early run reused a stale output directory, so an older binary-class split was overlaid on the five-class split, creating 624 duplicate images (385 shared between train and validation). The data-preparation script now wipes its output directory before writing; the split was regenerated and verified to have zero image overlap and instance counts matching the source. No contaminated number appears in this report.
- **GPU memory fixes the batch size.** Batch 8 caused a CUDA out-of-memory on the 8 GB card when training reached RT-DETR (the transformer peaks near 6.3 GB at batch 4). Batch 8 does not fit and batch 2 needlessly deviates from the paper, so all four models share the paper's intended batch 4, verified to fit.
- **Fork compatibility.** The vendored Ultralytics 8.0.201 does not run on modern PyTorch 2.8 and NumPy 2.4 unchanged. Six compatibility patches were applied (Appendix B); they are compatibility fixes only and change no model. Without them the DETR models silently never produced metrics, and an early "transformers fail" impression was in fact a CNN-only run.
- **Architecture correction and weight recovery.** The local `Diamond-detr.yaml` had been replaced with a non-paper FasterNet-T2 backbone variant (23.7 M parameters). The real paper architecture (`BasicBlock_Faster_Block_Rep` ResNet-18 variant, 14.4 M) was restored from a fresh clone of the source repository, which also yielded RT-DETR's diamond-pretrained checkpoint (39 MB, mAP@0.5 0.89) used as RT-DETR's init. No Diamond-DETR trained weights are published anywhere, so Diamond-DETR trains from random init.
- **The optimiser lesson.** An initial SGD run left both transformers near zero mAP. Switching to AdamW, the standard DETR-family optimiser, let RT-DETR converge from pretrained weights and Diamond-DETR converge from scratch (validation mAP@0.5 climbed from about 0.00001 to 0.80). This is the single most important methodological lesson of the project: the CNN defaults of SGD and COCO initialisation do not transfer to transformer detectors.

B Fork compatibility patches

The following six patches were applied in place to the vendored fork at `code/diamond_detr/RTDETR-main`. They restore compatibility with PyTorch 2.8 and NumPy 2.4 and fix one latent bug; none alter a model definition.

#	File	Fix
1	`utils/metrics.py`	`np.trapz` to `np.trapezoid` (NumPy 2.0 removed `trapz`); restores average-precision computation
2	`utils/torch_utils.py`	`torch.load(..., weights_only=False)` (PyTorch 2.6 changed the default to `True`)
3	`nn/tasks.py`	the same `weights_only=False` at two more call sites
4	`nn/tasks.py`	`attempt_load_one_weight` returns `(model, ckpt)`, not just `model` (latent fork bug)
5	`cfg/.../Diamond-detr.yaml`	restored the real paper architecture; keep the top-level `nc` key (its absence raises `KeyError`)
6	`nn/tasks.py`	`ConvNormLayer` fuse guard (`hasattr(m, 'norm')`) so a repeated `fuse()` from the FPS benchmark plus `val()` is a safe no-op

References

[1] Diamond-DETR. *Scientific Reports* (Nature), 2026. Source of the Diamond-DETR architecture and the diamond-domain reference figures.

[2] Y. Zhao et al. "DETRs Beat YOLOs on Real-time Object Detection." *CVPR*, 2024. arXiv:2304.08069.

[3] N. Carion et al. "End-to-End Object Detection with Transformers." *ECCV*, 2020.

[4] Y. Huang, C. Qiu, K. Yuan. "Surface defect saliency of magnetic tile." *The Visual Computer*, 36, 2020.

[5] G. Jocher et al. *Ultralytics YOLO* (YOLOv8, YOLO11). https://github.com/ultralytics/ultralytics.

[6] R. R. Selvaraju et al. "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization." *ICCV*, 2017.

[7] T.-Y. Lin et al. "Focal Loss for Dense Object Detection." *ICCV*, 2017.

[8] Y. Cui et al. "Class-Balanced Loss Based on Effective Number of Samples." *CVPR*, 2019.

[9] G. Ghiasi et al. "Simple Copy-Paste is a Strong Data Augmentation Method for Instance Segmentation." *CVPR*, 2021.

[10] K. Song, Y. Yan. "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects (NEU surface defect database)." *Applied Surface Science*, 2013.